

PRAKTIKUM INFORMATIK 2

M. Farmbauer, M. Niemetz

Zusammenwirkende Klassen

Wärmespeicherkomponenten

April 2026, Version: 3.2

1 Einführung

In dieser Aufgabe wird ein einfaches Simulationsmodell eines Kessels für eine Anlage zur Speicherung von Wärmeenergie erstellt. Dabei werden Klassen verwendet, um die Datenelemente und Funktionen zur Speicherung und Verwaltung der nötigen Daten sowie zur graphischen Darstellung des Kessels zu implementieren.

Konstruktoren und Destruktoren werden verwendet, um die Objekte bei der Erzeugung korrekt zu initialisieren und bei der Zerstörung der Objekte den durch ihr externes Eigentum belegten Speicherplatz wieder freizugeben.

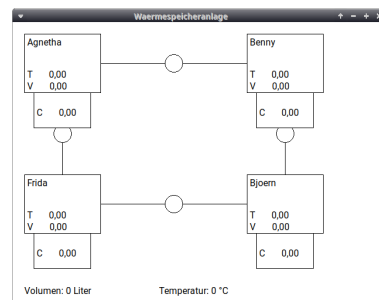


Abbildung 1: Darstellung der fertiggestellten Wärmespeicheranlage, wie sie nach dieser und den zwei folgenden Aufgaben aussehen wird. Zu erkennen sind die vier Kessel (große Rechtecke) mit ihren Brennern (kleine Rechtecke) und den Pumpen zum Transport des Mediums zwischen den Kesseln.

Sie werden die Energiespeicheranlage mittels Grafikfunktionen auf dem Bildschirm darstellen.

Hinweis: Lesen Sie die Angabe sorgfältig. Sollten Sie an einer Stelle nicht weiterkommen, lesen Sie die entsprechenden Themen in einem Buch oder dem Vorlesungsskript nach. Sprechen Sie bitte auch Ihre Dozenten an, falls Ihnen Erklärungen hier in der Aufgabenstellung fehlen.

Hinweis: Testen Sie unbedingt wie angegeben nach jeden Entwicklungsschritt Ihr Programm!

Hinweis: Im folgenden Teilabschnitt erhalten Sie Hinweise zu Fachwissen, das in der Aufgabe benötigt wird und Ihnen vielleicht noch nicht oder nicht mehr präsent ist.

Arbeiten Sie unbedingt vor Bearbeitung der Aufgabe, spätestens vor Beginn des betreffenden Abschnitts, die Punkte durch, sofern nötig.

Dadurch **ersparen Sie sich sehr viel Zeit** für **unnötiges Herumbasteln**.

Vorbereitung

- ☐ ☕☕ **Anleitung zu WinAdapt:** Machen Sie sich mit der Anwendung der Grafikbibliothek WinAdapt vertraut, indem Sie die Anleitung dazu studieren. Sie finden diese im ELearning-Kurs. 4
- ☐ ☕ **Analysieren Sie die bereitgestellte Klasse Point:** Verschaffen Sie sich einen Überblick über die Klasse Point um sie später nutzbringend einsetzen zu können. Dadurch können Sie viel Zeit und Ärger beim Bearbeiten der Aufgabe sparen. 5
- ☐ ☕☕ **Kennenlernen der Möglichkeiten der Klasse `string`:** Lesen Sie nach, was `string`-Objekte unterstützen und wie Sie damit arbeiten können. 6
- ☐ ☕ **Optionale Parameter:** Rekapitulieren Sie, wie Sie Methoden mit optionalen Parametern erzeugen können. Diese werden in der Literatur manchmal auch als „Default Parameter“ oder „Standardparameter“ bezeichnet. 7

2 Neues Projekt erstellen



Aktualisieren Sie ihren Fork des PIN2-Projekts, indem Sie `git pull upstream master` aufrufen. Überprüfen Sie anschließend, ob sich das Verzeichnis für diese Aufgabe nun in Ihrem Repository bzw. Arbeitsverzeichnis befindet.



Aktualisieren Sie ihren Fork des PIN2-Projekts.

3 Der Programmrahmen

3.1 Die Grafikumgebung „WinAdapt“

Da das Erstellen graphischer Anwendungen mit den hierfür verfügbaren leistungsfähigen Bibliotheken wie QT, GTK oder WxWidgets den Rahmen des Praktikums bei weitem sprengen würde, wird das notwendige Grundgerüst als kleine Bibliothek (WinAdapt) zur Verfügung gestellt.

WinAdapt enthält bereits das Hauptprogramm, welches das Fenster anzeigt und beim Auftreten gewisser Systemereignisse entsprechende vorgegebene *Hook-Funktionen* aufruft. In diese Hook-Funktionen müssen Sie Ihre auszuführenden Codesequenzen einfügen bzw. von dort aufrufen lassen, die bei den jeweiligen Systemereignissen ausgeführt werden sollen.

Die für Sie in der Datei `main.cpp` bereitgestellte Implementierung beinhaltet abgesehen von den Klassen den (fast) kompletten Code zur Darstellung der Wärmespeicheranlage. Da bisher noch keine Teile der Anlage

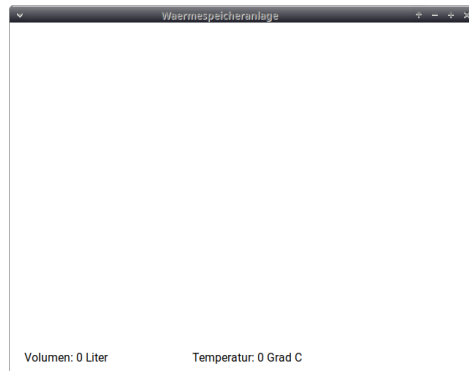


Abbildung 2: Screenshot des unveränderten Programmrahmens.

existieren, ist der meiste Code durch Präprozessordirektiven ausgeblendet.

Hinweis: Versuchen Sie nicht zum jetzigen Zeitpunkt alle Teile von `main.cpp` zu verstehen. Vieles kann sich erst später klären.

Wichtig: Das mitgelieferte Projekt unterstützt sowohl Windows-Systeme wie auch Linux-Systeme. Achten Sie darauf, das richtige *Build-Target* in CodeBlocks auszuwählen. Sie finden die Einstellung zum Beispiel im Menü unter *Build / Select Target*. Bei fehlerhafter Einstellung wird Ihr Projekt nicht compilieren.

Hinweis: Falls Sie Linux verwenden, benötigen Sie installierte GTK2+ Bibliotheken sowie die entsprechenden „devel“-Pakete, welche die nötigen Header-Dateien zur Bibliothek enthalten. Entsprechendes gilt auch für OS X.

Wichtig: Der Ursprung (Koordinate 0,0) liegt in der linken oberen Ecke des Bildschirmfensters. Damit wachsen die x-Koordinaten nach rechts und die y-Koordinaten nach unten.

Hinweis: Falls Sie eine andere Entwicklungsumgebung als CodeBlocks verwenden, stellen Sie bitte sicher, die notwendigen Bibliotheken zu Ihrem Programm hinzubinden zu lassen.

Hinweis: Eine Anleitung zur Benutzung von WinAdapt finden Sie im Dokument `WinAdapt.pdf` welches im Online-Kurs abgelegt ist.

Vor eigenen Änderungen testen Sie bitte unbedingt den Programmrahmen. Das Projekt sollte sich in dieser Phase problemlos übersetzen und starten lassen und beim Start ein relativ aufgeräumtes Bild wie in Abbildung 2 zeigen.

Recherche ☕☕

Anleitung zu WinAdapt



Testen Sie den Programmrahmen!

3.2 Eine erste Erweiterung

Wie Sie erkennen können, gibt das Programm eine Temperatur in °C und ein Volumen in L aus. Diese wird im weiteren Verlauf verwendet,



Implementieren Sie die Temperatur- und Volumenvorwahl und testen Sie Ihre Änderung.

um Speicherkessel mit dieser Menge an Flüssigkeit der angegebenen Temperatur zu befüllen.

Daher müssen Sie nun als erstes diese Vorwahlwerte bearbeitbar machen.

Die Tabelle 1 beschreibt die gewünschte Tastenbelegung.

Taste 1 ... 9	Vorwahl des Volumens für den nächsten Füllvorgang, 1→10 Liter, 2→20 Liter usw. Die vorgewählte Menge wird in einer globalen Variablen, <code>AmountRequested</code> , gespeichert.
Taste A ... Z	Vorwahl der Temperatur der Flüssigkeit für den nächsten Füllvorgang, A→5 °C, B→10 °C etc. Die vorgewählte Temperatur wird in einer globalen Variablen, <code>TemperatureRequested</code> , gespeichert. Wichtig: Hier sollen Großbuchstaben verwendet werden.

Tabelle 1: Funktion der Tasten zur Vorwahl von Volumen und Temperatur für Füllvorgänge.

Wenn man beispielsweise Kessel mit 30 Litern Flüssigkeit einer Temperatur von 25 Grad füllen will, so sind vorbereitend folgende Tasten zu drücken:

.

Zur Implementierung bietet Ihnen das WinAdapt-Framework folgende Möglichkeit an: Wird eine Taste auf der Tastatur gedrückt, so wird die Funktion `VtlKeyHit` aufgerufen und ihr die gedrückte Taste als Parameter übergeben. Dieser Parameter `key` enthält dabei den ASCII-Code der jeweils gedrückten Taste.

Hinweis: Um zum Beispiel den ASCII-Code einer Ziffer in den entsprechenden Zahlenwert umzuwandeln, kann man so vorgehen, dass der ASCII-Code der Ziffer Null, in C++ also `'0'`, vom übergebenen Parameterwert subtrahiert wird. Mit den Buchstaben kann man analog verfahren.

Hinweis: Überlegen Sie sich, wie der vorangehende Hinweis überhaupt funktioniert.

Implementieren Sie nun die Erweiterung der Funktion `VtlKeyHit`. Fügen Sie Ihren Code am Anfang der Funktion am vorgesehenen Platz ein. Achten Sie darauf, sonst keine Änderungen in der Datei vorzunehmen.

Testen Sie Ihre Änderung, indem Sie die Änderung der Bildschirmanzeige beim Betätigen der Tasten prüfen.



Erzeugen Sie nach dem erfolgreichen Test eine neue Revision und laden Sie das Ergebnis auf den Kursserver hoch.

3.3 Die Klasse `Point`

Bei der Programmierung graphischer Elemente ist es oft nötig, mit Bildschirmkoordinaten zu rechnen bzw. diese zu vergleichen. Dabei ist es praktisch, eine Klasse für den Umgang mit den Bildschirmkoordinaten zu implementieren, welche die wichtigsten Operationen bereitstellt. Dadurch wird die Implementierung des gewünschten Verhaltens nachher deutlich vereinfacht.

Um Ihnen die Arbeit zu erleichtern, haben wir für Sie eine entsprechende Klasse zusammen mit den Implementierungen für viele Operatoren bereitgestellt. Sie unterstützt zum Beispiel komponentenweise Addition, Multiplikation mit einem Skalar oder komponentenweise Vergleichsoperationen.

Frage: Die bereitgestellten Operatoren sind tatsächlich unerwartet mächtig. Überlegen Sie sich (am besten anhand einer kleinen Skizze), was es bedeutet, wenn für zwei `Point` Variablen `A`, `B` gilt, dass `A < B` wahr ist, oder dass `(B-A)<42.0` gilt.

Recherche ☕

Analysieren Sie die bereitgestellte Klasse `Point`

3.4 Die Klasse `string`

Neben der praktischen Klasse `Point` werden Sie in dieser Aufgabe auch die Klasse `string` aus der Standardbibliothek der Sprache C++ verwenden. Die Standardbibliothek ist eine sehr umfangreiche Sammlung an nützlichen Funktionen und Klassen¹.

Die Klasse `string` verwaltet Zeichenketten, kümmert sich dabei selbst um die Speicherverwaltung und unterstützt so sinnvolle Dinge wie eine Verkettung unterschiedlicher Zeichenketten über den Plusoperator zu einem neuen `string`-Objekt. Damit gehört die aus C bekannte mühsame (und fehlerträchtige) Verwaltung von Zeichenketten der Vergangenheit an – diesen Job hat jetzt `string`.

Für weitere Informationen zu `string` konsultieren Sie bitte die einschlägige Dokumentation, zum Beispiel [1].

Recherche ☕☕

Kennenlernen der Möglichkeiten der Klasse `string`.

¹Häufig liegen diese Funktionen und Klassen als Templates vor. Dies wird später behandelt und ist im Moment noch nicht so wichtig.

4 Klasse Boiler

Beginnen Sie nun mit der Implementierung der Klasse `Boiler`, welche in unserer Wärmespeicheranlage die Rolle eines Kessels wahr nimmt, der eine variable Menge einer Flüssigkeit mit variabler Temperatur enthalten kann.

Die hierfür nötigen Dateien `Boiler.h` und `Boiler.cpp` wurden bereits im Programmrahmen angelegt. Ebenso wurde die Deklaration der Klasse bereits für Sie in die Header-Datei eingetragen.



Implementieren Sie die Klasse `Boiler`.

Boiler
+ <code>ContentTemperature :double</code> + <code>ContainedVolume :double</code> + <code>Position :Point</code> + <code>Dimension :Point</code> + <code>Name :string</code>
+ <code>Boiler(Name :string Position :Point Dimension :Point ContainedVolume :double, ContentTemperature :double)</code> + <code>~Boiler()</code> + <code>addContent(MediaTemperature :double, MediaAmount : double) :void</code> + <code>addHeat(Amount :double) :void</code> + <code>show() :void</code>

Für die grafische Ausgabe muss der Kessel wissen, an welcher Stelle er auf dem Bildschirm erscheinen soll. Der Kessel benötigt deshalb Attribute in denen seine Position und seine Abmessungen gespeichert werden.

Neben den Koordinaten und den Abmessungen benötigt der Kessel außerdem ein Attribut zur Speicherung des Namens, damit dieser bei der Darstellung des Kessels mit ausgegeben werden kann.

Um seine Aufgabe als Speicher für Flüssigkeit einer bestimmten Temperatur wahrnehmen zu können, muss der Kessel außerdem noch Datenelemente zur Beschreibung dieser Eigenschaften des aktuellen Kesselinhalts bereit halten.

Zur Klasse `Boiler` gehören daher die folgenden Datenelemente:

Name	Typ	Beschreibung
<code>ContentTemperature</code>	<code>double</code>	Temperatur des Kesselinhalts.
<code>ContainedVolume</code>	<code>double</code>	aktuelles Flüssigkeitsvolumen im Kessel
<code>Position</code>	<code>Point</code>	Position des Kessels (linke obere Ecke) auf dem Bildschirm
<code>Dimension</code>	<code>Point</code>	Abmessungen des Kessels (Breite und Höhe)
<code>Name</code>	<code>string</code>	Name des Kessels für Bildschirmausgabe

Zudem soll die Klasse zunächst die Funktionen laut nachstehender Tabelle erhalten. Eine detaillierte Beschreibung finden Sie nach den Übersichtstabellen:

Recherche

Optionale Parameter

Name	Typ	Beschreibung
«Konstruktor»	—	Argumente des Konstruktors sind Name vom Typ <code>const string&</code> , <code>Position</code> und <code>Dimension</code> jeweils vom Typ <code>Point</code> , sowie die beiden <code>double</code> -Parameter <code>ContainedVolume</code> und <code>ContentTemperature</code> . Die letzten drei Argumente sollen optional sein und Defaultwerte (für <code>Dimension</code> : Breite 130 und Länge 100, für <code>ContainedVolume</code> und <code>ContentTemperature</code> jeweils 0.0) besitzen. Der Konstruktor soll alle Klassenattribute mit den Werten der Parameter initialisieren .
«Destruktor»	—	Der Destruktor hat keine Argumente und momentan noch nichts zu tun.

Name	Typ	Beschreibung
<code>addContent</code>	<code>void</code>	Zu der bereits im Kessel vorhandenen Flüssigkeitsmenge wird die als Parameter angegebene Menge hinzugefügt. Daher sind zwei Argumente vom Typ <code>double</code> nötig: <code>MediaTemperature</code> und <code>MediaAmount</code> für Temperatur und Menge der hinzugefügten Flüssigkeit.
<code>addHeat</code>	<code>void</code>	Führt dem Kessel die Menge an Wärmeenergie zu die im <code>double</code> -Argument übergeben wird, wodurch sich das Medium im Kessel erwärmt.
<code>show</code>	<code>void</code>	Stellt den Kessel auf dem Bildschirm dar.

Hinweis: Das Projekt kann in diesem Zustand noch nicht übersetzt werden. Hierfür ist erst die Implementierung der nächsten Teilaufgabe erforderlich.

Die Klasse `Boiler` wird später in verschiedenen Programmdateien verwendet. Daher sollen die Methoden nicht innerhalb der Klassendeklaration implementiert werden, sondern in die Klassendeklaration (.h-Datei) und die Implementierung der Methoden (.cpp-Datei) aufgeteilt werden, wie Sie dies von den vorangehenden Aufgaben bereits kennen.

4.1 Konstruktor und Destruktor

Nun wird nach den Vorgaben in obiger Tabelle ein Konstruktor erzeugt, mit dessen Hilfe beim Instantiieren (= Erzeugen von Variablen von einem Klassentyp) von `Boiler`-Objekten der Name und die Position festgelegt werden kann.

Am besten gibt man hier auch gleich den Füllstand und die Temperatur des Kessels mit an und kann diese so bequem vorbelegen.



Implementieren Sie den Konstruktor in der .cpp-Datei.



Implementieren Sie den Destruktor in der .cpp-Datei.

Ein Destruktor muss stets den im Konstruktor reservierten Speicher wieder freigeben. Da der Konstruktor derzeit keinen Speicher dynamisch belegt, muss (bzw. darf) der Destruktor im Moment auch noch nichts löschen.

Implementieren Sie den (folglich leeren) Destruktor und prüfen Sie, dann ob sich Ihr Programm fehlerfrei übersetzen lässt.



Erzeugen Sie nach dem erfolgreichen Test eine neue Revision und laden Sie das Ergebnis auf den Kursserver hoch.

4.2 Die Methode `addContent`

Für Methode `addContent()` ist es erforderlich, die Mischtemperatur T_{misch} zu berechnen, die sich aus der Temperatur T_1 und dem Volumen V_1 des Kesselinhalts sowie aus der Temperatur T_2 und dem Volumen V_2 der zugeführten Flüssigkeitsmenge ergibt.

Die Mischtemperatur T_{misch} berechnet sich hierbei mit Hilfe folgender Formel:

$$T_{\text{misch}} = \frac{V_1 T_1 + V_2 T_2}{V_1 + V_2}$$

Implementieren Sie die Methode `addContent()` und prüfen Sie, ob sich Ihr Programm fehlerfrei übersetzen lässt.



Erzeugen Sie nach dem erfolgreichen Test eine neue Revision und laden Sie das Ergebnis auf den Kursserver hoch.

4.3 Die Methode `addHeat`

Die Funktion `addHeat` besitzt einen Parameter vom Typ `double`, der angibt, welche Wärmemenge ΔE_W in der Einheit kJ dem Kessel zugeführt werden soll. Dabei verändert sich die Temperatur des Mediums im Kessel entsprechend der Beziehung:

$$T' = T + \frac{\Delta E_W}{V \cdot c_v}$$

mit der Wärmekapazität c_v des Mediums von $4,17 \text{ kJ L}^{-1} \text{ K}^{-1}$.



Implementieren Sie die Funktion `addContent` in der `.cpp`-Datei.



Implementieren Sie die Funktion `addheat` in der `.cpp`-Datei.

4.4 Die Methode show

Durch WinAdapt wird eine Funktion `Rect()` zur Verfügung gestellt, welche ein Rechteck mit horizontalen und vertikalen Kanten auf den Bildschirm zeichnet. Der Funktion `Rect()` müssen vier Parameter, nämlich jeweils die x- und y-Koordinaten der linken oberen sowie der rechten unteren Ecke, übergeben werden, wie folgendes Beispiel zeigt.

Beispiel:

```
Rect(X, Y, X+150, Y+150);
```

Das Rechteck ist in diesem Beispiel 150x150 Pixel groß.

Wichtig: Bei Verwendung von `Rect` ist es wichtig, dass das erste Koordinatenpaar die linke obere Ecke und das zweite Koordinatenpaar die rechte untere Ecke angibt. Sonst kann es auf manchen Systemen zu Fehlern kommen.

Hinweis: Selbstverständlich sollen die Kessel so gezeichnet werden, wie es ihre Attribute `Position` und `Dimension` angeben.

Zur Ausgabe des Kesselnamens wird die Funktion `Text()` verwendet, welcher die Startkoordinaten der Textausgabe sowie ein Zeiger auf eine C-Zeichenkette übergeben werden müssen. Dafür unterstützt die Klasse `string` die Methode `c_str()`.

Beispiel:

```
Text(X+10, Y+10, Name.c_str());
```

Jetzt muss die Methode `show()` noch um die Ausgabe des Füllstandes und der Temperatur erweitert werden. Hierfür muss die Ausgabe von Text und Zahlen gemischt in eine Zeichenkette erfolgen. Die C++ Standardbibliothek stellt hierfür eine Erweiterung Ihrer neuen Freundin, der `string`-Klasse, bereit. Diese unterstützt zusätzlich eine Ausgabe in einen String (statt auf ein Ausgabegerät) über den Operator `<<`.

Sie finden ein Beispiel für die Anwendung von `stringstream` am Ende der Funktion `VtlPaint` in `main.cpp`. Dort sehen Sie auch, wie der Inhalt der Zeichenkette zurückgesetzt werden kann.

Implementieren Sie die Funktion `show` und testen Sie, ob sich Ihr Programm korrekt und ohne Fehler übersetzen lässt.



Erzeugen Sie nach dem erfolgreichen Test eine neue Revision und laden Sie das Ergebnis auf den Kursserver hoch.



Zeichnen Sie die Kessel mit `show` in das Grafikfenster.



Ergänzen Sie eine Ausgabe für Temperatur und Füllstand.

5 Die Klasse Burner



Implementieren Sie die Methoden der Klasse **Burner**

Burner
– Name : string – Position : Point – MyBoiler : Boiler* – ConsumedFuel : double – ActualDimension : Point – ActualPosition : Point
+ Burner(MyBoiler : Boiler*, Name : string&, Position Point) + feedFuel(double Amount) : void + show() : void – updateCoordinates() : void

Die wie schon bereits **Boiler** bereits im vorbereiteten Projektrahmen angelegte Klasse **Burner** soll nun implementiert werden. Trennen Sie auch hier wieder zwischen Deklaration der Klasse (bereits vorbereitet) und der Implementierung der Funktionen. Die entsprechenden Dateien `Burner.cpp` und `Burner.h` sind bereits im Projekt vorhanden.

Die im UML-Diagramm links dargestellten Datenelemente und Funktionen werden im Folgenden ausführlich spezifiziert (in ihrer Funktionsweise und Schnittstelle beschrieben).

Die Klasse **Burner** erhält folgende Datenelemente:

- **Datenelemente**

Bekannt aus der **Boiler**-Implementierung:

Name	Typ	Beschreibung
Name	string	Name des Brenners für die Bildschirmausgabe
Position	Point	Soll-Position des Brenners auf dem Bildschirm (linke obere Ecke)

Neu für die Aufgaben der **Burner**-Objekte:

Name	Typ	Beschreibung
MyBoiler	Boiler*	Pointer auf das Boiler -Objekt, mit dem der Brenner physikalisch verbunden ist, dessen Inhalt er also heizen kann und mit dem er dafür mit ihm kommunizieren muss
ConsumedFuel	double	Verbrauchszähler, der die vom Brenner verfeuerte Menge an Brennstoff registriert

Für die angepasste Positionierung der **Burner**-Objekte:

Name	Typ	Beschreibung
<code>ActualDimension</code>	<code>Point</code>	Tatsächlich Abmessungen des Brenners (Breite und Höhe, berechnet)
<code>ActualPosition</code>	<code>Point</code>	Tatsächliche Position des Brenners (linke obere Ecke, berechnet)

• Methoden

- Ein **Konstruktor** mit drei Parametern (einer davon optional), nämlich `MyBoiler` des Typs Pointer auf `Boiler`, sowie `Name` des Typs `const string&` und dem optionalen Parameter `Position` des Typs `Point`. Wird das **optionale Argument für die Position** nicht angegeben, so soll der Wert `Point(-1, -1)` im entsprechenden Datenelement abgelegt werden.

Frage: Überlegen Sie sich, warum es günstig ist, die Parameter zur Namensübergabe als Typ `const string&` zu definieren. Diskutieren Sie die Frage mit Ihren Betreuern!

Der Konstruktor weist die übergebenen Parameter den entsprechenden Datenelementen zu und setzt den Zähler für den Brennstoffverbrauch auf 0.0. Die Datenelemente `ActualDimension` und `ActualPosition` müssen (weil sie als `Point`-Objekte nicht ohne Parameter konstruiert werden können, in der Initialisiererliste geeignet initialisiert werden (z.B. mit 0.0). Im ausführbaren Teil des Konstruktors können Sie dann durch Aufruf der Funktion `updateCoordinates` korrekt berechnet werden.

- `feedFuel` soll eine **void**-Methode mit einem **double** Parameter namens `Amount` sein. Der Parameter gibt die zu verbrennende Menge an **Brennstoff in Litern** an.

Beim Aufruf der Funktion `feedFuel` wird der Zähler für den verbrauchten Brennstoff um `Amount` erhöht und es wird die Funktion `addHeat` im zugehörigen Kessel unter Übergabe der Wärmemenge aufgerufen.

Wichtig: Diese Funktion soll auch in den Fällen korrekt „arbeiten“, in denen kein `Boiler`-Objekt mit dem `Burner`-Objekt verbunden ist. Nehmen Sie daher eine entsprechende Prüfung vor und führen Sie den Aufruf von `addHeat` nur dann vor, wenn auch ein gültiges `Boiler`-Objekt vorhanden ist.

Bei der Berechnung der erzeugten Wärmeenergie wird mit einem Heizwert des Brennstoffs (Erdgas) von **34,3 MJ pro Kubikmeter** gerechnet.

- `void show();`
Die Methode `show` zeichnet den Brenner unterhalb des zugehörigen Kessels und gibt den Namen und den Brennstoffverbrauch des Brenners aus.

Zur Ausgabe des Namens muss das Datenelement `Name` in eine Zeichenkette umgewandelt werden. Dazu verwendet man in bewährter Weise die Funktion `c_str()` der Klasse `string` welche direkt den benötigten Pointer auf die Zeichenkette zurückliefert.

Für die Position und die Abmessungen des Brenners werden die Datenelemente `ActualPosition` und `ActualDimension` herangezogen. Um sicherzustellen, dass diese aktuell sind, wird zu Beginn der Funktion `show` zunächst `updateCoordinates` aufgerufen.

- **`void updateCoordinates()` ;**

Die Darstellung des Brenners soll angepasst an den zugehörigen Kessel erfolgen, sofern der Brenner mit einem Solchen verbunden ist. Daher berechnet diese Funktion die tatsächliche Position `ActualPosition` und die tatsächlichen Abmessungen `ActualDimension` basierend auf Position und Abmessungen des zugehörigen Brenners.

Gehen Sie bei der Berechnung folgendermaßen vor:

- * **Falls kein Kessel verbunden ist:** Die Abmessungen des Brenners sollen 100x100 Punkte betragen und die x- bzw. y-Koordinate soll null sein, falls die entsprechende Koordinate in `Position` kleiner als Null ist. Andernfalls wird der Wert für die jeweilige Koordinate aus `Position` übernommen.

- * **Falls ein Kessel verbunden ist:** Die Breite des Brenners soll 75% der Breite des verbundenen Kessels betragen. Die Höhe soll 25% der Kesselhöhe, aber mindestens 60 Punkte betragen.

Die Position des Brenners `ActualPosition` wird für Koordinaten, die ≥ 0 sind aus dem Datenelement `Position` übernommen. Negative Koordinatenwerte werden nicht übernommen sondern es werden Ersatzwerte so berechnet, dass der Brenner mittig unmittelbar unterhalb des Kessels platziert wird.

Wichtig: Das Datenelement `Position` soll nicht verändert werden. Insbesondere sollen negative Koordinatenwerte erhalten bleiben und nicht durch die berechneten Werte ersetzt werden.

Nach dem Erstellen der Funktionen der Klasse `Burner` sollte getestet werden, ob sich das Projekt mit den neu hinzugekommenen Inhalten fehlerfrei übersetzen lässt.



Erzeugen Sie nach dem erfolgreichen Test eine neue Revision und laden Sie das Ergebnis auf den Kursserver hoch.

Taste	Aktion
; :	Befüllen von Kessel 1 nach Vorwahl befüllen von Kessel 2 nach Vorwahl
, .	Zuführen von Wärme zum Kessel 1 Zuführen von Wärme zum Kessel 2
Space	Zuführen von Brennstoff zu Brenner 0
1 ... 9	Vorwahl des Volumens
A ... Z	Vorwahl der Temperatur

Tabelle 2: Tastaturbelegung, die für den Testbetrieb im aktuellen Modus `STAGE == CREATE_COMPONENTS` gültig ist.

6 Der Test

Nun ist es an der Zeit, den Programmcode nicht nur durch Compilieren syntaktisch prüfen zu lassen, sondern den Code auch ausführen zu lassen um in diesem dynamischen Test eventuelle Fehler erkennen und beheben zu können.

Ändern Sie dafür in der Datei `Stages.h` den Wert der Präprozessorkonstante `STAGE` auf den Wert `CREATE_COMPONENTS` ab und compilieren Sie Ihr Programm. Es sollte problemlos compilieren und eine Bildschirmdarstellung wie in Abbildung 3 zeigen.

Analysieren Sie die über den Präprozessor mit der Einstellung `STAGE == CREATE_COMPONENTS` in der Datei `main.cpp` aktivierten Programmteile.

Sie können auch bereits mit den dargestellten Komponenten über die in Tabelle 2 dargestellte Tastaturbelegung interagieren.

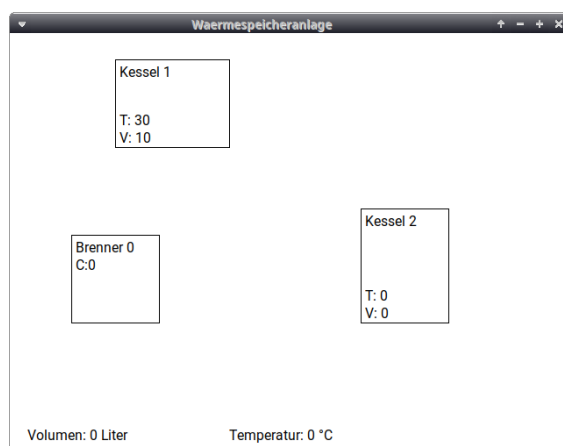


Abbildung 3: Ausgabe des Testprogramms auf dem Bildschirm



Führen Sie einen dynamischen Test Ihrer neu entwickelten Funktionen durch.



Analysieren Sie die über den Präprozessor mit der Einstellung `STAGE == CREATE_COMPONENTS` aktivierten Programmteile.



Beheben Sie eventuelle Fehler in Ihrem Programmcode.

Beheben Sie eventuelle Fehler in Ihrem Programmcode.



Erzeugen Sie nach dem erfolgreichen Test eine neue Revision und laden Sie das Ergebnis auf den Kursserver hoch.

Die Entwicklung an der Wärmespeicheranlage wird in der nächsten Aufgabe fortgeführt.

Literatur

- [1] Cplusplus.com: C++ Reference—The `string` Class,
<https://cplusplus.com/reference/string/string/?kw=string>